MODULE BIG DATA ANALYTICS

Projet Capstone

Apache Spark

SparkSQL · SparkML · Spark Streaming · GraphX

Cahier des charges de projets sur données réelles

Hmida HMIDA

1. Présentation générale

1.1 Contexte et finalités

Ce projet a pour but de mettre en application, sur des données réelles et volumineuses, l'ensemble des compétences acquises : analyse structurée avec **SparkSQL**, apprentissage automatique avec **SparkML**, traitement de flux avec **Spark Streaming** et analyse de graphes avec **GraphX**.

Chaque équipe choisit l'un des cinq projets décrits dans ce document. Chaque projet part d'un **problème concret**, s'appuie sur un **jeu de données public** dont le lien est fourni, et se décline en objectifs précis pour chacune des quatre composantes de Spark. L'objectif n'est pas seulement de faire fonctionner des algorithmes, mais de construire une **chaîne de traitement complète**, justifiée, mesurée et reproductible, de l'ingestion de la donnée brute jusqu'à la restitution des résultats.

1.2 Compétences visées

Composante	Compétences évaluées
SparkSQL	DataFrames et Datasets, schémas explicites, jointures, fonctions de fenêtre, agrégations avancées, formats colonnaires (Parquet), partitionnement, lecture des plans d'exécution (explain), optimisation.
SparkML	Pipelines, transformers et estimators, ingénierie de variables, gestion du déséquilibre de classes, sélection de modèle (CrossValidator, TrainValidationSplit), métriques adaptées, sauvegarde et rechargement des modèles.
Spark Streaming	Structured Streaming : sources et sinks, fenêtres temporelles, watermark, agrégations avec état, checkpoint, tolérance aux pannes, application d'un modèle sur un flux.
GraphX	Graphe de propriétés (sommets et arêtes), opérateurs structurels, aggregateMessages, Pregel, algorithmes standards (PageRank, composantes connexes, comptage de triangles, propagation d'étiquettes) et exploitation de leurs résultats dans un modèle.
Transversal	Architecture de données, qualité et nettoyage, reproductibilité, travail en équipe avec Git, communication des résultats.

1.3 Exigence commune : le noyau minimal

Quel que soit le projet, le rendu doit démontrer l'usage **effectif** des quatre composantes. Le noyau minimal exigé est le suivant :

- **SparkSQL** : ingestion avec schéma explicite, nettoyage, au moins huit requêtes analytiques commentées, stockage en Parquet partitionné.
- **SparkML** : au moins un Pipeline complet, deux algorithmes comparés à une baseline simple, une validation adaptée au problème (découpage temporel si les données sont horodatées) et un modèle sauvegardé.
- **Spark Streaming** : une application Structured Streaming avec fenêtres temporelles, watermark et checkpoint, qui consomme un flux réel ou rejoué.
- **GraphX** : un graphe construit à partir des données du projet, au moins deux algorithmes appliqués et l'interprétation de leurs résultats.

2. Cadre technique et méthodologique commun

2.1 Environnement

- **Apache Spark 3.x ou 4.x**, en mode local pour la mise au point puis sur cluster (Docker Compose, cluster pédagogique ou plateforme cloud) pour les volumes complets.
- **Langages** : PySpark pour SparkSQL, SparkML et Structured Streaming. La partie graphe se code avec GraphFrames en Python.
- **Apache Kafka** (ou une source fichiers rejouée) pour alimenter les applications de streaming.
- **Git** pour le code ; un README permettant à un tiers de reproduire l'ensemble du pipeline.

2.2 Architecture de référence en couches

Couche	Contenu	Format et stockage
Brute (bronze)	Données telles que reçues, sans transformation, avec schéma explicite.	CSV, JSON ou Parquet d'origine ; copie immuable
Nettoyée (silver)	Types corrigés, valeurs aberrantes traitées, doublons supprimés, jointures de référence.	Parquet partitionné (par date, par zone, etc.)
Analytique (gold)	Agrégats, tables de variables (features), graphes prêts à l'emploi.	Parquet ; tables temporaires ou persistantes
Modèles ML	PipelineModel sauvegardés, paramètres et métriques de chaque expérience.	Répertoires de modèles Spark ; fichier de suivi des expériences
Temps réel	Applications Structured Streaming, états et points de reprise.	Kafka, Parquet, console ; répertoire de checkpoint

2.3 Bonnes pratiques attendues

- Définir les **schémas explicitement** plutôt que de se reposer sur l'inférence automatique.
- Contrôler le **partitionnement** (nombre de partitions, repartition ou coalesce, éviter les petits fichiers) et utiliser cache ou persist de façon justifiée.
- Éviter collect et toPandas sur de gros volumes.
- Découper les données en **entraînement, validation et test** selon la chronologie lorsque les enregistrements sont horodatés, afin d'éviter toute fuite d'information du futur vers le passé.
- Fixer les graines aléatoires, versionner le code et consigner les paramètres de chaque expérience.

Projet 1 — NYC Mobility : anticiper la demande de taxis en temps réel

Thème	Mobilité urbaine et transport
Volume	Plusieurs millions de courses par mois ; au moins 12 mois à traiter (fichiers Parquet mensuels)

Contexte

La Taxi & Limousine Commission (TLC) de la ville de New York publie chaque mois les enregistrements détaillés des courses de taxis jaunes, de taxis verts et de véhicules de transport avec chauffeur. Chaque enregistrement décrit les heures et les zones de prise en charge et de dépose, la distance, le montant détaillé de la course, le mode de paiement et le nombre de passagers.

Problème à résoudre

Dans ce scénario, un opérateur de flotte souhaite répartir ses véhicules au plus près de la demande. Ses chauffeurs se déplacent aujourd'hui à l'intuition, ce qui génère des kilomètres à vide et des temps d'attente pour les clients.

Vous devez construire une plateforme capable (1) de comprendre la demande passée, (2) de la prévoir par zone et par tranche horaire, (3) de la suivre en temps réel et (4) de signaler les écarts anormaux entre la demande observée et la demande prévue.

Données

Jeu de données	Description, format et volume	Lien
TLC Trip Record Data	Courses des taxis jaunes et verts et des véhicules de location avec chauffeur. Publication mensuelle au format Parquet, avec dictionnaires de données, table de correspondance des zones (taxi zone lookup) et fichier de formes des zones.	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
Miroir Azure Open Datasets (taxis jaunes)	Les mêmes courses au format Parquet, organisées par année et par mois ; pratique pour un accès programmatique.	https://learn.microsoft.com/en-us/azure/open-datasets/dataset-taxi-yellow

- Les fichiers sont publiés avec un retard d'environ deux mois. Les liens de téléchargement mensuels se trouvent sur la page de la TLC.
- Le schéma évolue d'un mois à l'autre (colonnes et frais ajoutés) : définir un schéma explicite et documenter les écarts.
- La TLC indique que ces données n'ont pas été produites par elle et qu'elle n'en garantit pas l'exactitude : prévoir une vraie phase de nettoyage (dates incohérentes, distances nulles, montants négatifs).
- Commencer par 2 à 3 mois pour la mise au point, puis passer à 12 mois ou plus.

Objectifs

Objectif général. Construire un pipeline complet — ingestion, analyse, modélisation, scoring en temps réel — qui prévoit le nombre de courses par zone et par heure et détecte les écarts anormaux.

SPARKSQL

- Charger au moins 6 mois de courses avec un schéma explicite et stocker une version nettoyée en Parquet partitionné par année et par mois.
- Joindre la table des zones pour obtenir l'arrondissement et le nom de chaque zone, puis construire une table d'agrégats « courses par zone et par heure ».
- Écrire au moins dix requêtes analytiques (fonctions de fenêtre, rollup ou cube, top-N par zone, saisonnalité jour/semaine, effet des jours fériés.

SPARKML

- Prédire le nombre de courses par zone et par heure (régression) à partir de variables calendaires, de la zone et de la demande des heures précédentes (variables retardées), au sein d'un Pipeline.
- Construire un second modèle pour prédire le pourboire ou la durée d'une course.
- Comparer au moins trois algorithmes (régression linéaire, forêt aléatoire, arbres boostés) avec validation croisée ; reporter RMSE, MAE et R^2 face à une baseline naïve, par exemple la moyenne historique par zone et par heure.

SPARK STREAMING

- Rejouer les courses d'un mois comme un flux (source fichiers avec limitation par micro-lot, ou Kafka).
- Calculer la demande par zone sur des fenêtres glissantes de 15 minutes avec watermark.
- Appliquer le modèle sauvegardé au flux et lever une alerte lorsque la demande observée dépasse la prévision d'un seuil à définir ; écrire les résultats dans un sink avec checkpoint.

GRAPHX

- Modéliser les zones comme sommets et les flux de courses (zone de départ vers zone d'arrivée) comme arêtes pondérées.
- Calculer PageRank, degrés pondérés et communautés pour identifier les zones « hubs » ; injecter ces indicateurs comme variables supplémentaires du modèle ML et mesurer le gain obtenu.

Livrables spécifiques

- Jeu de données nettoyé (Parquet) accompagné d'un rapport de qualité : taux de lignes rejetées par règle de nettoyage.
- Dix requêtes SQL avec interprétation métier de chaque résultat.
- Modèles sauvegardés et tableau comparatif des algorithmes.
- Application Structured Streaming démontrable, avec un suivi « demande observée contre demande prévue » par zone.
- Export du graphe des zones (sommets et arêtes) pour visualisation, avec commentaire des zones les plus centrales.

Critères de réussite

- Le meilleur modèle améliore nettement la baseline en RMSE (objectif indicatif : au moins 15 %).
- Le traitement streaming suit le rythme du flux : le temps de traitement d'un micro-lot reste inférieur à son intervalle de déclenchement.
- Le pipeline se rejoue de bout en bout à partir du README.

Points de vigilance

- Découper entraînement et test **chronologiquement** : un découpage aléatoire ferait fuiter le futur dans l'entraînement.
- Traiter les zones inconnues et les valeurs aberrantes sans fausser les agrégats.
- Surveiller le nombre de fichiers et de partitions produits lors des écritures.

Projet 2 — CinéReco : un moteur de recommandation de films

Thème	Systèmes de recommandation, plateformes de contenu
Volume	25 millions de notes, 1 million d'applications de tags, 62 000 films, 162 000 utilisateurs

Contexte

Les plateformes de vidéo à la demande doivent aider chaque utilisateur à trouver, parmi des dizaines de milliers de titres, ceux qui lui plairont. Le jeu de données MovieLens, collecté et maintenu par le groupe de recherche GroupLens de l'université du Minnesota, est une référence pour évaluer les algorithmes de recommandation.

Problème à résoudre

Dans ce scénario, une plateforme constate que ses utilisateurs abandonnent la navigation lorsqu'ils ne trouvent pas de contenu pertinent en quelques minutes.

Vous devez concevoir un moteur de recommandation qui (1) prédit les notes qu'un utilisateur donnerait à des films qu'il n'a pas vus, (2) propose une liste personnalisée de N films, (3) gère le démarrage à froid pour les films peu ou pas notés et (4) réagit en continu aux nouvelles notes.

Données

Jeu de données	Description, format et volume	Lien
MovieLens 25M	Archive ml-25m.zip (environ 250 Mo) : notes, films, tags et « tag genome » (15 millions de scores de pertinence sur 1 129 tags). Fichiers CSV : ratings, movies, tags, genome-scores, genome-tags, links.	https://grouplens.org/datasets/movielens/25m/
Autres versions MovieLens	Versions réduites (latest-small, 100K, 1M) pour le développement local.	https://grouplens.org/datasets/movielens/

- Mettre au point le code sur une version réduite, puis lancer l'entraînement final sur la version 25M.
- La version 25M ne contient pas de données démographiques : les utilisateurs ne sont connus que par leur identifiant.

Objectifs

Objectif général. Livrer un moteur de recommandation évalué rigoureusement, avec un mode batch (entraînement) et un mode flux (tendances et recommandations mises à jour).

SPARKSQL

- Explorer les données : distribution des notes, longue traîne de popularité, biais des utilisateurs (indulgents ou sévères), évolution dans le temps.

- Extraire l'année depuis le titre, éclater les genres, calculer les meilleurs films par genre et par décennie avec des fonctions de fenêtre, et exploiter les tags.
- Construire les jeux d'entraînement, de validation et de test par découpage chronologique.

SPARKML

- Entraîner un modèle de filtrage collaboratif ALS, régler rank, regParam et maxIter par validation, et gérer correctement les utilisateurs et films absents de l'entraînement (stratégie de démarrage à froid).
- Évaluer par RMSE sur les notes, puis par des métriques de classement (precision@k, NDCG@k) face à une baseline de popularité.
- Traiter le démarrage à froid avec une approche par contenu : vecteurs du tag genome, similarité cosinus ou clustering K-Means ; proposer une combinaison hybride.

SPARK STREAMING

- Simuler l'arrivée de nouvelles notes en rejouant les données horodatées.
- Calculer par fenêtre glissante les films « tendance » (top-N) et la note moyenne glissante avec un nombre minimal de votes.
- Joindre le flux aux facteurs latents du modèle ALS (jointure flux-statique) pour produire des recommandations actualisées pour les utilisateurs actifs.

GRAPHX

- Construire un graphe biparti utilisateurs-films pondéré par la note, ou un graphe de films reliés par co-notation.
- Appliquer PageRank, composantes connexes et propagation d'étiquettes pour détecter des communautés de goûts ; comparer ces communautés aux genres officiels.

Livrables spécifiques

- Rapport d'exploration (SQL) avec visualisations et enseignements pour le métier.
- Modèle ALS sauvegardé, tableau des réglages testés et courbe des métriques.
- Fonction de démonstration recommande(utilisateur, N) accompagnée d'une explication des résultats pour deux profils contrastés.
- Application de streaming des tendances et des recommandations actualisées.
- Étude comparative des communautés détectées par GraphX et des genres.

Critères de réussite

- Le RMSE sur le jeu de test est inférieur à celui des baselines (moyenne globale, moyenne par film).
- La precision@10 dépasse celle d'une recommandation par simple popularité.
- Le démarrage à froid est traité et son effet est mesuré.

Points de vigilance

- Un découpage aléatoire des notes surestime la performance : privilégier un découpage temporel.

- Les modèles de type ALS peuvent produire des valeurs non définies pour les éléments inconnus à l'entraînement.
- La génération de recommandations pour tous les utilisateurs est coûteuse en mémoire : travailler sur des sous-ensembles lors de la mise au point.
- Attention au biais de popularité, qui favorise toujours les mêmes films.

Projet 3 — WikiWatch : surveillance en temps réel de l'activité de Wikipédia

Thème	Analyse de flux, surveillance de communautés en ligne
Volume	Flux continu de modifications de tous les projets Wikimedia ; archive à constituer par l'équipe (quelques heures selon ressources)

Contexte

Les projets de la fondation Wikimedia diffusent en continu chaque modification effectuée sur leurs pages : création, édition, journalisation d'actions. Ce flux public, disponible via le service EventStreams, représente des milliers d'événements par minute répartis sur des centaines de wikis.

Problème à résoudre

Dans ce scénario, l'équipe de modération d'une communauté souhaite repérer plus vite les situations qui demandent l'attention de contributeurs : pics d'activité sur une page (actualité brûlante, conflit d'édition), comportements automatisés, comptes agissant de concert.

Vous devez construire une chaîne de surveillance qui ingère le flux en continu, calcule des indicateurs en temps réel, apprend à distinguer les comportements automatisés des contributions humaines, et met en évidence des groupes de comptes qui éditent les mêmes pages.

Données

Jeu de données	Description, format et volume	Lien
Flux des modifications récentes (recentchange)	Flux d'événements au format JSON, diffusé en continu (Server-Sent Events) pour tous les wikis publics.	https://stream.wikimedia.org/v2/stream/recentchange
Documentation EventStreams	Description du service, des flux disponibles et exemples de clients.	https://wikitech.wikimedia.org/wiki/Event_Platform/EventStreams
Présentation du flux des modifications récentes	Explication du fonctionnement du flux côté MediaWiki.	https://www.mediawiki.org/wiki/API:Recent_changes_stream
Schéma JSON des événements	Définition des champs de chaque événement de modification récente.	https://github.com/wikimedia/mediawiki-event-schemas/blob/master/jsonschema/mediawiki/recentchange/1.0.0.yaml

- Il n'existe pas de fichier à télécharger : l'équipe doit **construire sa propre ingestion**, par exemple un client qui lit le flux et publie les événements dans Kafka, que Spark consomme ensuite.
- Archiver les événements en Parquet pendant une durée suffisante pour disposer d'un jeu d'entraînement et d'un graphe de co-édition.
- Respecter les conditions d'usage du service : un seul client par équipe, avec un en-tête User-Agent identifiant le projet, et reconnexion propre en cas de coupure.

Objectifs

Objectif général. Mettre en place une chaîne de surveillance en temps réel tolérante aux pannes, qui produit des indicateurs, des alertes et une analyse a posteriori des comportements d'édition.

SPARK STREAMING (CŒUR DU PROJET)

- Consommer le flux depuis Kafka, analyser le JSON avec un schéma explicite et gérer les événements mal formés.
- Calculer des agrégats sur fenêtres : modifications par minute et par wiki, répartition par type d'événement, part des contributions automatisées.
- Détecter avec un traitement à état les pages qui dépassent un nombre de modifications dans une fenêtre de 10 minutes, ainsi que les comptes très actifs.
- Démontrer la tolérance aux pannes : arrêter puis relancer l'application et prouver la reprise sans perte ni doublon grâce au checkpoint.

SPARKSQL

- Sur l'archive Parquet, produire des analyses : wikis les plus actifs, heures de pointe, répartition automatisé/humain, taille des modifications (différence de longueur avant/après).
- Comparer les agrégats calculés en streaming et ceux obtenus en batch sur la même période.

SPARKML

- Entraîner un classifieur qui distingue les modifications automatisées (champ indiquant un robot) des modifications humaines, à partir de variables non triviales : wiki, type d'événement, espace de noms, variation de taille, heure, caractéristiques du titre.
- Construire un détecteur d'anomalies par K-Means sur des vecteurs d'indicateurs par minute, en considérant comme anormaux les points éloignés de leur centroïde.
- Charger le modèle dans l'application de streaming pour scorer le flux.

GRAPHX

- À partir de l'archive, construire un graphe biparti comptes-pages, puis sa projection : graphe de comptes reliés lorsqu'ils éditent les mêmes pages.
- Calculer composantes connexes, comptage de triangles et PageRank ; identifier les groupes de comptes fortement liés et en proposer une interprétation prudente.

Livrables spécifiques

- Code du client d'ingestion et configuration Kafka.

- Application Structured Streaming avec checkpoint et démonstration de reprise après arrêt.
- Archive Parquet des événements collectés et rapport SQL associé.
- Classifieur et détecteur d'anomalies avec métriques.
- Analyse du graphe de co-édition avec les groupes de comptes identifiés.

Critères de réussite

- L'application tourne de façon continue sans intervention.
- La reprise après arrêt n'entraîne ni perte ni doublon dans les agrégats.
- Le classifieur est évalué avec des métriques adaptées au déséquilibre éventuel des classes, et comparé à une règle simple.

Points de vigilance

- Le nom du compte ne doit pas servir de variable explicative : il rendrait la classification triviale et non généralisable.
- Les événements arrivent en retard ou dans le désordre : choisir un watermark justifié.

Projet 4 — ExpertRank : cartographie du réseau d'entraide de Stack Overflow

Thème	Analyse de réseaux sociaux et de communautés d'entraide
Volume	2,6 millions d'utilisateurs et 63,5 millions d'interactions horodatées sur environ 2 774 jours

Contexte

Stack Overflow est un espace d'entraide où des millions de développeurs posent des questions, y répondent et commentent les contributions des autres. Le groupe SNAP de l'université Stanford en a extrait un réseau temporel d'interactions : chaque arête relie deux utilisateurs à un instant donné.

Trois types d'interactions sont distingués : un utilisateur répond à la question d'un autre (a2q), commente la question d'un autre (c2q) ou commente la réponse d'un autre (c2a). Le graphe global est l'union des trois.

Problème à résoudre

Dans ce scénario, les responsables de la communauté veulent identifier les contributeurs qui structurent réellement l'entraide, comprendre comment les échanges s'organisent et anticiper le départ des membres précieux.

Vous devez analyser le réseau à grande échelle, mesurer l'influence des utilisateurs, repérer des sous-communautés et prédire quels utilisateurs resteront actifs.

Données

Jeu de données	Description, format et volume	Lien
sx-stackoverflow (SNAP)	Fichiers sx-stackoverflow.txt.gz (toutes interactions), sx-stackoverflow-a2q, sx-stackoverflow-c2q, sx-stackoverflow-c2a. Format texte « SRC DST UNIXTS » : un utilisateur source, un utilisateur destination, un horodatage Unix. 2 601 977 sommets, 63 497 050 arêtes temporelles.	https://snap.stanford.edu/data/sx-stackoverflow.html
Autres réseaux Stack Exchange (SNAP)	Même format, plus petits : Math Overflow (environ 25 000 sommets), Super User (environ 194 000 sommets), Ask Ubuntu. Idéal pour la mise au point.	https://snap.stanford.edu/temporal-motifs/data.html
Super User (SNAP)	Page de détail du réseau Super User, utile comme jeu intermédiaire.	https://snap.stanford.edu/data/sx-superuser.html

- Mettre au point les algorithmes sur Math Overflow ou Super User avant de passer au graphe complet de Stack Overflow.

- Le graphe est un **multigraphe** : plusieurs interactions peuvent relier les mêmes utilisateurs. Décider et justifier la façon de les agréger (comptage, poids, dédoublonnage).
- Les identifiants de nœuds correspondent à l'identifiant de propriétaire dans l'export de données de Stack Exchange.
- Implémenter un algorithme avec **Pregel** ou aggregateMessages, par exemple une propagation d'influence ou un plus court chemin depuis un expert.

Objectifs

Objectif général. Caractériser la structure et la dynamique du réseau d'entraide, classer les utilisateurs par influence et prédire leur rétention en combinant graphe, SQL temporel et apprentissage.

GRAPHX (CŒUR DU PROJET)

- Charger les arêtes et construire le graphe de propriétés ; calculer degrés entrants et sortants, et en étudier la distribution.
- Appliquer PageRank (avec critère de convergence justifié), composantes connexes, composantes fortement connexes, comptage de triangles et propagation d'étiquettes.
- Comparer les structures obtenues pour les trois types d'interactions (a2q, c2q, c2a) et interpréter les différences.

SPARKSQL

- Analyser la dynamique temporelle : interactions par mois et par type, utilisateurs actifs par trimestre, part d'utilisateurs ponctuels, taux de réciprocité.
- Construire par SQL les profils temporels des utilisateurs : date de première et de dernière activité, nombre de jours actifs, volume par type d'interaction.

SPARKML

- Prédire si un utilisateur sera encore actif sur une période future : calculer les variables sur une période passée, puis l'étiquette sur la période suivante (découpage temporel strict).
- Combiner variables de profil et variables issues du graphe (degrés, PageRank, appartenance à une composante) ; mesurer l'apport du graphe en comparant avec et sans.
- Comparer régression logistique, forêt aléatoire et arbres boostés ; évaluer par AUC et courbe précision-rappel.

SPARK STREAMING

- Rejouer les arêtes dans l'ordre chronologique par micro-lots, en simulant un temps événementiel accéléré.
- Maintenir en continu, sur fenêtres glissantes, le nombre de réponses reçues par utilisateur et afficher le classement des « experts du moment ».

Livrables spécifiques

- Code Scala de chargement et d'analyse du graphe, exécutable sur le cluster.
- Tableau des utilisateurs les plus influents avec comparaison des mesures (degré, PageRank).

- Rapport sur la structure du réseau : composantes, communautés, comparaison des trois types d'interactions.
- Modèle de rétention avec évaluation et analyse de l'apport des variables de graphe.
- Application de classement en continu des experts du moment.

Critères de réussite

- Le graphe complet est chargé et analysé sur le cluster avec une consommation mémoire justifiée.
- Les résultats de PageRank sont comparés au simple degré entrant, avec une discussion argumentée.
- Le modèle de rétention est évalué sur une période strictement postérieure à celle des variables.

Points de vigilance

- Ne pas utiliser pour l'étiquette une information qui a servi à construire les variables (fuite d'information).
- Surveiller la taille des partitions et les déséquilibres de données (quelques utilisateurs concentrent un très grand nombre d'arêtes).
- Interpréter avec prudence : une interaction n'est pas une preuve d'expertise.

Projet 5 — AML-Chain : détection de transactions Bitcoin illicites

Thème	Lutte contre le blanchiment et la fraude financière
Volume	203 769 transactions (sommets), 234 355 flux (arêtes), 166 variables, 49 pas de temps

Contexte

La société Elliptic, spécialisée dans l'analyse de la blockchain pour la lutte contre la criminalité financière, a publié un graphe de transactions Bitcoin où chaque nœud est une transaction et chaque arête un flux de fonds d'une transaction vers une autre. Les transactions sont réparties sur 49 pas de temps successifs.

Une partie des transactions est étiquetée « licite » (échanges, portefeuilles, mineurs, services financiers) ou « illicite » (escroqueries, logiciels malveillants, rançongiciels, etc.) ; environ 2 % sont illicites, 21 % licites et 77 % restent sans étiquette.

Problème à résoudre

Dans ce scénario, la cellule de conformité d'une plateforme d'échange doit repérer les transactions suspectes avant de les accepter. Les analystes humains ne peuvent examiner qu'un faible nombre de cas par jour : le modèle doit donc être très précis sur la classe illicite tout en n'en laissant pas passer trop.

Vous devez construire un détecteur de transactions illicites, mesurer l'apport de la structure du graphe par rapport aux seules variables des transactions, puis l'exploiter comme un système d'alerte rejouant le temps réel.

Données

Jeu de données	Description, format et volume	Lien
Elliptic Data Set (Kaggle)	Trois fichiers CSV : elliptic_txs_features (variables), elliptic_txs_classes (étiquettes : illicite, licite, inconnue) et elliptic_txs_edgelist (arêtes). Compte Kaggle gratuit nécessaire pour le téléchargement.	https://www.kaggle.com/datasets/ellipticco/elliptic-data-set
Extension optionnelle : Elliptic++	Article décrivant une version enrichie (adresses de portefeuilles et variables supplémentaires), pour les équipes qui souhaitent aller plus loin.	https://arxiv.org/pdf/2306.06108
Article de référence sur les distances aux nœuds illicites	Travail exploitant la distance aux nœuds illicites du graphe Elliptic : source d'idées de variables de graphe.	https://arxiv.org/pdf/2102.05373

- Les variables sont anonymisées : les 94 premières décrivent la transaction elle-même, les suivantes agrègent l'information des transactions voisines.
- Le jeu est **très déséquilibré** et majoritairement non étiqueté : l'entraînement et l'évaluation ne portent que sur les nœuds étiquetés.
- Le jeu de données reste modeste en volume : l'intérêt du projet est méthodologique ; pour tester le passage à l'échelle, l'équipe pourra répliquer artificiellement les données.

Objectifs

Objectif général. Construire un détecteur de transactions illicites évalué honnêtement (découpage temporel, métriques sur la classe minoritaire) et démontrer l'apport des variables extraites du graphe.

SPARKML (CŒUR DU PROJET)

- Entraîner une classification binaire licite/illicite sur les nœuds étiquetés, avec un **découpage temporel** (par exemple pas de temps 1 à 34 pour l'entraînement et 35 à 49 pour le test).
- Traiter le déséquilibre : pondération des classes, sous-échantillonnage ou sur-échantillonnage, réglage du seuil de décision.
- Comparer régression logistique, forêt aléatoire et arbres boostés ; évaluer par précision, rappel, F1 et aire sous la courbe précision-rappel **sur la classe illicite**, et non par l'exactitude globale.
- Mesurer le gain apporté par les variables de graphe : variables locales seules, puis variables locales et variables de graphe.

GRAPHX (CŒUR DU PROJET)

- Construire le graphe orienté des transactions à partir des fichiers de variables et d'arêtes.
- Calculer des variables de structure : degrés entrant et sortant, PageRank, composante connexe, nombre de triangles.
- Utiliser aggregateMessages pour agréger les caractéristiques des voisins, et Pregel pour calculer la distance au plus proche nœud illicite connu.
- Étudier si les transactions illicites forment des regroupements dans le graphe.

SPARKSQL

- Joindre variables, étiquettes et arêtes ; analyser la répartition des classes par pas de temps et le niveau d'étiquetage.
- Produire un rapport de qualité et une table de variables prête pour la modélisation, stockée en Parquet.

SPARK STREAMING

- Rejouer les pas de temps 35 à 49 comme un flux, un pas de temps par micro-lot.
- Scorer chaque transaction avec le PipelineModel sauvegardé, générer des alertes classées par score et suivre la précision et le rappel pas de temps par pas de temps.

Livrables spécifiques

- Table de variables (locales et de graphe) en Parquet et rapport d'analyse exploratoire.

- Modèles sauvegardés et tableau comparatif avec et sans variables de graphe.
- Code GraphX des variables de structure et de la distance aux nœuds illicites.
- Application de scoring en continu et tableau de suivi des performances par pas de temps.
- Note de synthèse destinée à un analyste de conformité : forces, limites et seuils recommandés.

Critères de réussite

- Le meilleur modèle améliore nettement le rappel de la classe illicite à précision comparable à celle d'une baseline simple.
- Le gain des variables de graphe est quantifié, discuté et illustré par des exemples.
- Les performances sont rapportées pas de temps par pas de temps, y compris les dégradations éventuelles.

Points de vigilance

- Ne jamais utiliser les étiquettes des nœuds de test pour construire les variables de voisinage : c'est une fuite d'information classique dans les problèmes de graphes.
- Éviter les découpages aléatoires : les comportements frauduleux évoluent, et les performances se dégradent souvent sur les pas de temps les plus récents.
- Les nœuds sans étiquette ne sont pas des nœuds licites : ne pas les traiter comme tels.

Organisation, livrables et évaluation

Livrables communs

- **Dépôt Git** contenant le code, le rapport et la présentation. Le dépôt contient un README de reproduction et la description de l'architecture.
- **Rapport** de 15 à 20 pages : problème, données, démarche, résultats, limites et pistes d'amélioration.
- **Démonstration** du pipeline de bout en bout, y compris l'application de streaming.
- **Soutenance** de 15 minutes suivie de 5 minutes de questions, avec présence de tous les membres.